

# EECE\_5699 Computer Hardware and System Security

## Homework 3

Sakthivel P. Sivakumar

NUID: 002312294

## Differential Fault Analysis of AES-128

### Overview:

The goal of this assignment was to simulate a single byte fault during AE-128 encryption and use DFA to recover the affected 4-byte column of the last-round subkey (round 10)

This Implementation has 2 parts, first is to write a simulator code that generate pairs (C, F) for the same plaintext where F is produced with a controlled one-byte XOR fault at a known state location right before MixColumns in round 9.

Next is the Analysis part, in which from multiple pairs with the same fault location and  $\Delta$ , I had to recover the 4 key bytes for the corresponding last-round column; and show how candidate spaces shrink and estimate how many pairs are needed.

### Part 1: On-line (Fault Simulation)

- The target cipher is AES-128, which consists of 10 rounds.
- The fault (single byte XOR) is induced during Round 9. It is injected into the 4x4 internal state after the SubBytes and ShiftRows operations of Round 9, but before the MixColumns operation.
- The fault value is an unknown, non-zero byte  $\Delta$ , where  $\Delta$  is in the range[1,255].
- It affects exactly one byte at a known coordinate (row, col) within the 4x4 state matrix.
- The single byte fault at (row, col) is immediately processed by the Round 9 MixColumns operation. This causes the single-byte difference to propagate to all four bytes of that specific column (col) at the output of the MixColumns step.
- The attack continues normally through the remainder of Round 9 (AddRoundKey) and all of Round 10 (SubBytes, ShiftRows, AddRoundKey).
- This process results in two ciphertexts: the correct ciphertext and the faulty ciphertext. The difference between the correct and faulty ciphertexts (their XOR sum) has a specific, structured pattern that can be analyzed to deduce information about the round key.

## Part 2: Off-line (Fault Analysis)

- For each of the four rows (r) in the affected column, the relationship between the clean and faulty ciphertexts, after inverting the final SubBytes operation, is:
- Due to the final round's ShiftRows operation, the ciphertext column index j holding the data from the internal state row r (for the targeted column col) is found using the relation:  $j = (col - r) \bmod 4$ .

## Entire DFA Implementation Run:

```
PS D:\M.S. in ECE\Fall 2025\EECE_CHSS_HW3> python "d:\M.S. in ECE\Fall 2025\EECE_CHSS_HW3\aes\fault_dfa.py"
Showing target last-round key bytes (row, col, value):
  row 0, col 2 -> 0x21
  row 1, col 1 -> 0xa6
  row 2, col 0 -> 0xdd
  row 3, col 3 -> 0xaf
After 1 pair: remaining  $\Delta = 15$ ; per-row candidate sizes = [(2, 2), (2, 2), (2, 2), (2, 4)]
After 2 pairs: remaining  $\Delta = 1$ ; per-row candidate sizes = [(1, 1), (1, 1), (1, 1), (1, 1)]
Solved with  $\Delta = 0x7c$ 
  key byte at (row 0, col 2) is 0x21
  key byte at (row 1, col 1) is 0xa6
  key byte at (row 2, col 0) is 0xdd
  key byte at (row 3, col 3) is 0xaf

Recovering with provided pairs.

Batch trials: seeing how many pairs we are needing on average:
Trials: 20
Pairs needed – min/median/max: 2 / 2 / 2
PS D:\M.S. in ECE\Fall 2025\EECE_CHSS_HW3>
```

## Extra Credit 1:

In this part, I recovered all 4 columns (entire round-10 key) by repeating DFA per column.

### Run Results:

```
PS D:\M.S. in ECE\Fall 2025\EECE_CHSS_HW3> python "d:\M.S. in ECE\Fall 2025\EECE_CHSS_HW3\aes\extra_credit1.py"
Recovered last-round key (round 10):
  a9 c9 05 98
  14 40 d8 03
  27 38 d8 f7
  f5 52 eb 47
True last-round key (from AES):
  a9 c9 05 98
  14 40 d8 03
  27 38 d8 f7
  f5 52 eb 47
```

## Extra Credit 2:

In this part, I inverted the AES-128 key schedule from K10 to recover the original 16-byte master key.

### Run Results:

```
PS D:\M.S. in ECE\Fall 2025\EECE_CHSS_HW3> python "d:\M.S. in ECE\Fall 2025\EECE_CHSS_HW3\aes\extra_credit2.py"
Recovered master key: 7b26dde3e50f3224ea6c4a1a87f37f9
True master key      : 7b26dde3e50f3224ea6c4a1a87f37f9
Round-10 key matches: True
PS D:\M.S. in ECE\Fall 2025\EECE_CHSS_HW3>
```

## Conclusion:

- Simulated an AES-128 single-byte XOR fault at the round-9 MixColumns input to produce (C, F) pairs.
- Ran DFA to recover the affected 4-byte last-round subkey column, printing candidate shrinkage and pairs used.
- Used fault\_dfa.py to implement simulator, analyzer, and batch trials (min/median/max pairs)
- Extra Credit 1 (extra\_credit1.py) recovered the full round-10 key by solving each column.
- Extra Credit 2 (extra\_credit2.py) inverted the key schedule to recover the original 16-byte master key